Spec-driven development with GitHub Spec Kit

These instructions take an idea from a vague problem to working, tested code in this repo using GitHub Spec Kit. You check whether the idea is worth building, write a specification, settle the open questions, plan the change, break it into tasks, and only then let the agent write code against the plan.

Spec Kit works with the coding agent of your choice. The examples use **GitHub Copilot CLI**; see Using another agent for Codex and Claude Code.

Install

1. Prerequisites

Tool	Why	Check
.NET 10 SDK	Build, test and run the app	`dotnet --version` (10.x)
Node.js 22.20 or later	Repo helper scripts	`node --version`
Git	Branch per step so you can compare and roll back	`git --version`
uv	Installs and runs the Spec Kit CLI; it fetches Python 3.11+ automatically	`uv --version`
A coding agent	Runs the Spec Kit steps: Copilot CLI, Codex or Claude Code	`copilot --version`

2. Check the app builds and its tests pass

```
dotnet test
dotnet run --project src/CRUDTasksWithAgent   # http://localhost:5280
```

Note what the task list does today; it's the "before" picture.

3. Install the Spec Kit CLI

```
uv tool install specify-cli
specify version
```

If you'd rather not install it, prefix each `specify` command with `uvx --from specify-cli` .

4. Initialise Spec Kit in this repo, on a branch

```
git switch -c speckit/my-feature
specify init --here --force --integration copilot --script sh   # Windows: --script ps
```

- `--here` initialises the current folder.
- `--force` is needed because the folder isn't empty. It merges Spec Kit's files into the repo and may overwrite files with the same names, so run it on a branch and review `git status` afterwards.
- It adds `.specify/` (templates, scripts, the constitution) and one skill per step in `.github/skills/speckit-*/` .

5. Add the idea-assessment extension

```
specify extension add assess
```

This adds the five `speckit-assess-*` skills used in steps 2–6 below.

6. Turn on automatic commits

```
specify extension add git
uv run --with pyyaml python scripts/configure_speckit_git.py .
```

After this, every step commits itself with a fixed message such as `[Spec Kit] Add specification`, and specify creates a feature branch (`NNN-<feature>`). Nobody has to remember to commit, and you can compare or roll back any step. On another repo, copy `scripts/configure_speckit_git.py` across first. docs/speckit-setup.md explains exactly what this changes and why.

7. Check your agent can see the skills

Start your agent in the repo folder and ask it:

```
List the skills available to you whose names start with speckit.
```

You should see the `speckit-*`, `speckit-assess-*` and `speckit-git-*` skills. If not, restart the agent from the repo root.

Commit at this point (`git add -A && git commit -m "Install Spec Kit"`) so the setup is separate from the feature work. From here on, the steps commit themselves.

> **On branch `demo/01-init`, install steps 3–6 are already done.** See docs/speckit-setup.md for how it was configured.

The steps

Run the steps **in your agent's chat, one at a time**, and review the file each one writes before moving on. Each step writes a Markdown file that the next step reads, so you can start a fresh session between steps without losing context. Each step also **commits itself**; glance at `git log --oneline` to confirm, and run `/speckit-git-commit` if one is missing.

> **Want the exact prompts?** Walkthrough: copy and paste every step has a complete, ready-to-paste prompt for each step below, worked through on this repo.

#	Step (Copilot / Claude)	What it does	W
1	`/speckit-constitution <principles>`	Records the project's non-negotiable rules, e.g. testing, layering, dependencies, lint. **Run once per project.**	`.specify/memo`
2	`/speckit-assess-intake slug=<name> <idea>`	Captures a rough idea (text, ticket or pointer into the code) as a normalised intake note.	`.specify/asse`
3	`/speckit-assess-research slug=<name>`	Gathers evidence for and against the idea: users, prior art, existing code and data.	`…/<slug>/rese`
4	`/speckit-assess-define slug=<name>`	Defines the problem: who is affected, what hurts, goals, non-goals and success metrics.	`…/<slug>/prob`

#	Step (Copilot / Claude)	What it does	W
5	`/speckit-assess-shape slug=<name>`	Sketches solution options, scope, appetite and trade-offs, with no implementation design.	`…/<slug>/conc`
6	`/speckit-assess-decide slug=<name>`	Gives a verdict, **go**, **needs-clarification** or **kill**, and for a go, a handoff for specify.	`…/<slug>/deci`
7	`/speckit-specify use the handoff summary in .specify/assessments/<slug>/decision.md`	Writes the feature spec: user stories, acceptance scenarios and requirements, with **no implementation detail**. Marks unknowns `[NEEDS CLARIFICATION]`.	`specs/NNN-<fe` `checklists/re`
8	`/speckit-clarify`	Asks up to five targeted questions about gaps in the spec and writes the answers back into it.	Updates `spec.`
9	`/speckit-plan <tech constraints>`	Designs the change against the existing code and the constitution: research, data model, API contracts and a quickstart.	`plan.md`, `rese` `contracts/`, `q`
10	`/speckit-checklist` *(optional)*	Generates a custom quality checklist for the feature, e.g. for UX, security or API.	`checklists/<t`
11	`/speckit-tasks`	Breaks the plan into small, dependency-ordered tasks, marking which can run in parallel.	`tasks.md`
12	`/speckit-analyze` *(optional)*	Read-only consistency check across the spec, plan and tasks: gaps, contradictions and breaches of the constitution.	A report in chat
13	`/speckit-implement`	Works through `tasks.md`, writing code and tests and ticking off tasks.	Source and test `tasks.md`

#	Step (Copilot / Claude)	What it does	W
14	`/speckit-converge`	Compares the code with the spec, plan and tasks, and adds any unbuilt work to `tasks.md` as new tasks.	Appends to `tas`

Afterwards, verify outside the agent. The compiler, tests and linter are the real judges:

```
dotnet test
node scripts/lint-count.mjs --check
dotnet run --project src/CRUDTasksWithAgent
```

Optional: to turn `tasks.md` into GitHub Issues, run `specify extension add github`, then `/speckit-github-taskstoissues`. This needs a GitHub remote and the GitHub MCP server.

Walkthrough: copy and paste every step

A complete worked example for this repo. Each block is a full prompt: copy it, paste it into your agent's chat, wait for the step to finish, review what it wrote, then move on. The example idea is "*users can't tell what to work on first*", which leads to backlog item **B1 "Due dates and priority"** in docs/backlog.md.

- **Codex users:** change the leading `/` to `$` (e.g. `$speckit-constitution`). Everything else is the same.
- **Your own repo:** the steps are the same. Rewrite the constitution for your codebase and replace the idea with your own.

Before you start

```
git switch -c run/walkthrough-$(date +%Y%m%d-%H%M%S)   # a fresh branch per attempt; steps 1–6 commit before Spec Kit creates its fea
# PowerShell: git switch -c "run/walkthrough-$(Get-Date -Format yyyyMMdd-HHmmss)"
copilot                           # or: codex / claude
```

After each step, run `git log --oneline -3` in another terminal. You should see a new `[Spec Kit] …` commit. If one is missing, run `/speckit-git-commit`.

Step 1: Constitution (once per project)

What it does: records the project's non-negotiable rules (tests, layering, dependencies, lint, offline). Every later step is checked against them.
Why: standards get enforced on every step instead of depending on someone remembering them.

```
/speckit-constitution Create the constitution for this .NET 10 Blazor Interactive Server task app. Principles:
1. Every behaviour change ships with xUnit tests in tests/CRUDTasksWithAgent.Tests, at the service, API and app-agent tool levels as
2. Keep the layering: Model -> TaskService -> minimal API endpoints in Program.cs -> Blazor page. App-agent tools in Tools/TaskCrudT
3. Any new or changed app-agent tool needs a matching scripted intent in Services/Scripted/ScriptedIntents.cs, with a working Example
4. Do not add build warnings beyond the baseline in docs/lint-baseline.md; check with node scripts/lint-count.mjs --check.
5. No new NuGet packages without a written justification in the plan.
6. The app must stay runnable offline with no keys (the scripted model is the default).
7. The UI stays Blazor Interactive Server; moving to WebAssembly is out of scope unless a spec says otherwise.
```

Review: `.specify/memory/constitution.md`. Every principle should be checkable, not vague.

Step 2: Assess, intake

What it does: captures the raw idea (the problem as it arrived, plus any evidence) as a normalised intake note. No solution yet.
Why: starts from the real problem before anyone commits to an answer.

```
/speckit-assess-intake slug=what-first Users say they can't tell what to work on first. The task list is one flat list in creation o
```

Review: `.specify/assessments/what-first/intake.md` .

> The assess steps (2–6) don't commit on their own, so `.specify/assessments/what-first/` shows as untracked in `git status` . That's expected: it's committed as `[Spec Kit] Add idea assessment` when you run Specify (step 7).

Step 3: Assess, research

What it does: gathers evidence for and against the idea from the repo and what you've given it: prior art, the current code, constraints.
Why: tests the idea against evidence rather than enthusiasm, and finds work that already exists (here, backlog item B1).

```
/speckit-assess-research slug=what-first Look for prior art and constraints in this repo: docs/backlog.md (especially B1), the TaskI
```

Review: `research.md` . It should cite B1 and the current data model.

Step 4: Assess, define

What it does: defines the problem: who it affects, what hurts, goals, non-goals and how success will be measured.
Why: you can't judge a solution without an agreed problem and a way to tell it worked.

```
/speckit-assess-define slug=what-first Keep the problem focused on individuals deciding what to do next. A good success metric: with
```

Review: `problem.md` : problem, users, goals, non-goals, success metrics.

Step 5: Assess, shape

What it does: sketches solution options with their trade-offs, recommends one, and sets an appetite (how much it's worth spending). No implementation design.
Why: decides the scope and the budget before anyone designs or builds.

```
/speckit-assess-shape slug=what-first Compare at least two options, e.g. priority only, versus due date plus priority with sort and
```

Review: `concept.md` : options, trade-offs, the recommended option and the appetite.

Step 6: Assess, decide

What it does: gives a verdict (**go**, **needs-clarification** or **kill**). For a go, it writes a handoff summary for specify.
Why: only ideas worth building reach delivery. A documented kill is a good outcome.

```
/speckit-assess-decide slug=what-first
```

Review: `decision.md` gives **go**, **needs-clarification** or **kill**. A "go" commits the assessment (`[Spec Kit] Add idea assessment`) when you run the next step. For **needs-clarification**, edit the file it names (e.g. add evidence to `research.md`) and run this step again.

Step 7: Specify

What it does: turns the approved concept into a feature spec: user stories, acceptance scenarios and requirements, with **no implementation detail**. Unknowns are marked `[NEEDS CLARIFICATION]` . It also creates a feature branch.
Why: agrees *what* and *why* before *how*. The spec is the contract the finished code is checked against.

```
/speckit-specify Use the handoff summary in .specify/assessments/what-first/decision.md to specify the approved concept.
```

This creates and switches to a feature branch such as `001-what-first` .

Review: `specs/001-*/spec.md` : user stories, acceptance scenarios, and `[NEEDS CLARIFICATION]` markers. No implementation detail.

Step 8: Clarify

What it does: finds gaps and ambiguities in the spec, asks you up to five targeted questions, and writes the answers back into the spec.
Why: a human settles the ambiguities now, instead of the agent guessing halfway through coding.

```
/speckit-clarify Where these decisions answer a question, apply them; ask me about anything else:
- Priority levels are Low, Normal and High; new tasks default to Normal.
- Due dates are dates only (no time); "today" uses the server's local date.
- A task is overdue when its due date is before today and it is not complete.
- Default sort: priority (High first), then due date (earliest first, no date last), then title.
- Filters for priority, due before a date, and overdue can be combined.
- The chat agent can set and change priority and due date, through the existing tools plus scripted intents.
- An invalid priority or date in an API request returns 400.
```

Review: `spec.md` has a "Clarifications" section, and the `[NEEDS CLARIFICATION]` markers are gone. Answer any extra questions it asks.

Step 9: Plan

What it does: designs the change against the existing code and the constitution: technical research, data model, API contracts and a quickstart.
Why: the approval gate. You review and approve *how* before any code is written.

```
/speckit-plan Use the existing stack only: EF Core in-memory via TasksDbContext, minimal API endpoints in Program.cs, Blazor Interac
```

Review and approve before any code is written: `plan.md`, `research.md`, `data-model.md`, `contracts/` and `quickstart.md`. Check the constitution check passes.

Step 10: Checklist (optional)

What it does: generates a quality checklist for the areas you name (e.g. API, UX, security), to test the spec and plan against.
Why: turns "is this good enough?" into explicit items you can check.

```
/speckit-checklist Create a checklist for API and UX quality: validation and 400 responses, sort and filter behaviour, keyboard acces
```

Review: `checklists/`. Tick or fix items before tasks.

Step 11: Tasks

What it does: breaks the plan into small, ordered tasks, with tests first and parallel tasks marked.
Why: small tasks keep each change reviewable and give the agent a script to follow.

```
/speckit-tasks Write tests first for each layer (service, API, app-agent tool), then the implementation, then the UI. Keep each task
```

Review: `tasks.md`. Tasks should be small, ordered, and each traceable to a requirement.

Step 12: Analyze (optional)

What it does: a read-only consistency check across the spec, plan and tasks: gaps, contradictions and breaches of the constitution. It changes no files.
Why: catches drift between the documents before it becomes code.

```
/speckit-analyze
```

Review: the report in chat. Fix any contradictions between the spec, plan and tasks before implementing.

Step 13: Implement

What it does: works through `tasks.md`, writing code and tests and ticking tasks off as it goes.
Why: the agent builds against an approved plan instead of improvising.

```
/speckit-implement Work through tasks.md in order. After each phase, run dotnet test and fix failures before continuing. At the end,
```

Review: the diff, and `tasks.md` with tasks ticked off.

Step 14: Converge

What it does: compares the code with the spec, plan and tasks, and adds any unbuilt work to `tasks.md` as new tasks.
Why: closes the loop. Repeat implement and converge until it reports **Converged**.

```
/speckit-converge
```

If it adds tasks, run `/speckit-implement` again, then `/speckit-converge`, until it reports **Converged**.

Check the result yourself

```
dotnet test
node scripts/lint-count.mjs --check
dotnet run --project src/CRUDTasksWithAgent --urls http://localhost:5280
curl "http://localhost:5280/api/tasks"      # tasks now include priority and due date
```

Open http://localhost:5280/ to try the new controls, and http://localhost:5280/agentfx to see the chat's new suggested prompts. Then compare the result with the success metric from step 4.

Using another agent

The steps, files and order are the same in every agent. Only the setup flag and how you invoke a step differ:

	Copilot CLI	Codex	Claude Code
`specify init … --integration`	`copilot`	`codex` (needs the `codex` CLI, or add `--ignore-agent-tools`)	`claude`
Skills are installed in	`.github/skills/speckit-*`	`.agents/skills/speckit-*`	`.claude/skills/speckit-*`
Invoke a step	`/speckit-specify …`	`$speckit-specify …`	`/speckit-specify …`
Switch an initialised repo	`specify integration switch copilot`	`specify integration switch codex`	`specify integration switc`

Run `specify integration list` to see every supported agent.

On branch `demo/01-init`, Copilot and Codex are both set up, so Codex users can type `$speckit-…` straight away. To add a second agent to your own repo, see docs/speckit-setup.md.